Journal of Hardware and Systems Security (2026) 10:4
https://doi.org/10.1007/s41635-026-00177-1

RESEARCH

TroLLoc: Security Closure against Hardware Trojans by Logic Locking and Layout Hardening

Fangzhou Wang[1] · Qijing Wang[1] · Lilas Alrahis[2] · Bangqi Fu[1] · Shui Jiang[1] · Xiaopeng Zhang[1] · Ozgur Sinanoglu[3] · Tsung-Yi Ho[1] · Evangeline F. Y. Young[1] · Johann Knechtel[3]

Received: 1 March 2025 / Accepted: 18 February 2026
© The Author(s) 2026, modified publication 2026

Abstract

Due to cost benefits, supply chains of integrated circuits (ICs) are largely outsourced nowadays. However, passing ICs through various third-party providers gives rise to many security threats, like piracy of IC intellectual property or insertion of hardware Trojans, i.e., malicious circuit modifications. In this work, we proactively and systematically protect the physical layouts of ICs against post-design insertion of Trojans. Toward that end, we propose *TroLLoc*, a novel scheme for IC security closure that employs, in careful unison, logic locking and layout hardening, i.e., physical synthesis aimed toward highest possible utilization. *TroLLoc* is fully integrated into a commercial-grade design flow, and shown to be effective, efficient, and robust. Our work provides in-depth layout and security analysis considering the ISPD'22/23 benchmarks for security closure. *TroLLoc* successfully renders layouts resilient, with reasonable overheads, against (i) general prospects for Trojan insertion as in the ISPD'22 contest, (ii) actual Trojan insertion as in the ISPD'23 contest, and (iii) second-order attacks where adversaries would first (before Trojan insertion) try to bypass the locking defense, mainly using advanced machine learning attacks. Finally, we release all our artifacts for independent verification [64].

Keywords Integrated circuits · Hardware security · Trojans · Logic locking · Physical design · Security closure

1 Introduction

Integrated circuits (ICs) of varying scale and complexity are at the heart of all our modern-day information systems. An ever-growing body of security threats can compromise information systems in general and ICs in particular [31, 35, 43, 53]. For the design, manufacturing, and deployment of ICs, there are numerous companies and partners involved within complex and world-wide supply chains—ICs run through many hands, where some of those may be acting with malicious intent. Furthermore, once ICs are deployed in the field, adversaries can employ powerful hands-on measures, targeting directly at the hardware and software at runtime.

The resulting cybersecurity challenges are manifold, suggesting the need for holistic and streamlined defense efforts, all the way from software applications down to the IC hardware. Toward the latter end, electronic design automation (EDA) algorithms and tools will play an important role [31, 36, 52, 58]. However, state-of-the-art (SOTA) commercial EDA tools do not account yet for such security challenges. **Security Closure** is an emerging paradigm that seeks to address this shortcoming. The idea is to proactively harden the layouts of ICs, during physical synthesis, against various threats that are executed post design-time [17, 37, 38, 41]. Such efforts are important for multiple reasons. For example, threats like hardware Trojans or side-channel attacks are explicitly exploiting vulnerabilities of the IC layouts, and any such threats that are not mitigated during

✉ Fangzhou Wang
fzwang@cse.cuhk.edu.hk

Lilas Alrahis
lilas.malrahis@ku.ac.ae

Johann Knechtel
johann@nyu.edu

[1] Department of Computer Science and Engineering, Chinese University of Hong Kong, Hong Kong, Hong Kong

[2] Department of Computer and Information Engineering, Khalifa University, Abu Dhabi, UAE

[3] Center for Cyber Security, New York University Abu Dhabi, Abu Dhabi, UAE

Published online: 01 April 2026

design-time will be very difficult to address later on—unlike software, ICs are not easily patchable. See Sec. 2.2 for more background.

In this work, we focus on the threat of **hardware Trojans**, i.e., malicious circuit modifications that can be introduced by different adversaries throughout the IC supply chain. By design, Trojans are only minor in extent but severe in fallout [15, 76, 77]. Many different types of countermeasures have been proposed over the years. However, most prior art falls short in terms of effectiveness, efficiency, and/or robustness. See Sec. 2.1 for more background on Trojans in general and Sec. 3 for a discussion of relevant prior art, respectively. Besides, **logic locking**, or locking for short, means to incorporate so-called key-gate structures that are controlled by secret key-bits [79]. Without knowing the related key-bits required to unlock those key-gate structures, attackers cannot fully understand the structure and functionality of the design at hand. While locking is mainly known for protection of chip-design intellectual property (IP), it can also serve for Trojan defense. See Sec. 2.3 for more background on locking.

The **objective of this work** is security closure against hardware Trojans. That is, we aim for *proactive, pre-silicon Trojan prevention, by carefully and systematically hardening the physical layout of ICs against post-design Trojan insertion*. Based on a thorough review of prior art and its limitations, we derive important **research challenges** as follows. Note that various concepts indicated on here, like second-order attacks or open placement sites, are all explained in detail in Sec. 2.

- *Robustness – Any Trojan defense must remain robust in place.* That is, a foundry-based adversary, being fully aware that some defense is put in place, would naturally want to first circumvent that defense (i.e., stealthily remove, override, or otherwise render useless) before inserting their Trojans. Such second-order attacks represent a key challenge where prior art falls short.
- *Effectiveness – Any defense should protect against various Trojans.* This is especially true for proactive, pre-silicon defenses which have only "one shot" at design-time. More specifically, designs should be protected as a whole, i.e., in terms of (i) layout resources exploitable for Trojan insertion in general (i.e., open placement sites, free routing tracks, available timing slacks) and (ii) the netlist structure and functionality, which is relevant to hinder Trojans targeting on specific assets. Furthermore, defenses should be evaluated against actual layout-level Trojan insertion, not only against vulnerability estimates.
- *Efficiency – Any defense should incur limited overheads.* Taking control over the underlying trade-offs for resilience against Trojans versus design overheads requires two parts: (i) metrics for security and overheads, and (ii) some integrated, secure-by-design EDA methodology, utilizing those metrics and some user guidance.

To address all these challenges, our work makes the following **contributions**. A high-level representation of our contributions is also given in Fig. 1.

1. Layout-level logic locking – We propose a novel locking scheme, called *TroLLoc*, devised to hinder post-design Trojan insertion. Together with the IC security-closure flow, *TroLLoc* meets important challenges for Trojan defenses: it is robust, effective, and efficient. See further below for an explanation of these achievements.
2. IC security-closure flow – We devise an EDA flow for design-time security closure against post-design Trojan insertion. The flow serves to implement the proposed *TroLLoc* scheme and is fully integrated into a commercial-grade design setup.
3. Layout and security analysis – We conduct a set of thorough case studies that show the promises of the proposed scheme. Toward that end, we are using the real-world benchmark suites from the recent ISPD'22/23 contests on security closure [17, 38, 40]. We are employing a set of SOTA attacks on locking, namely *MuxLink* [5], *OMLA* [6], *Resynthesis* [4], and *SCOPE* [3], and we also conduct actual Trojan insertion as in [17, 40, 41], both to demonstrate the resilience of the proposed scheme against advanced Trojan attacks.
4. Release – We release our secured layouts in [64], to enable independent verification of our work.

In short, our work is (i) robust, (ii) effective, and (iii) efficient. For (i), we show that advanced adversaries employing (a) second-order attacks using SOTA machine learning (ML)-based attacks on locking and/or (b) actual Trojan insertion are hindered. For (ii), we harden IC layouts against Trojan insertion in general and further protect security-sensitive assets in particular. For (iii), we employ both locking and layout hardening in unison, along with commercial-grade tooling for full control on the security-vs-overhead trade-offs in a real-world IC design and manufacturing setting.

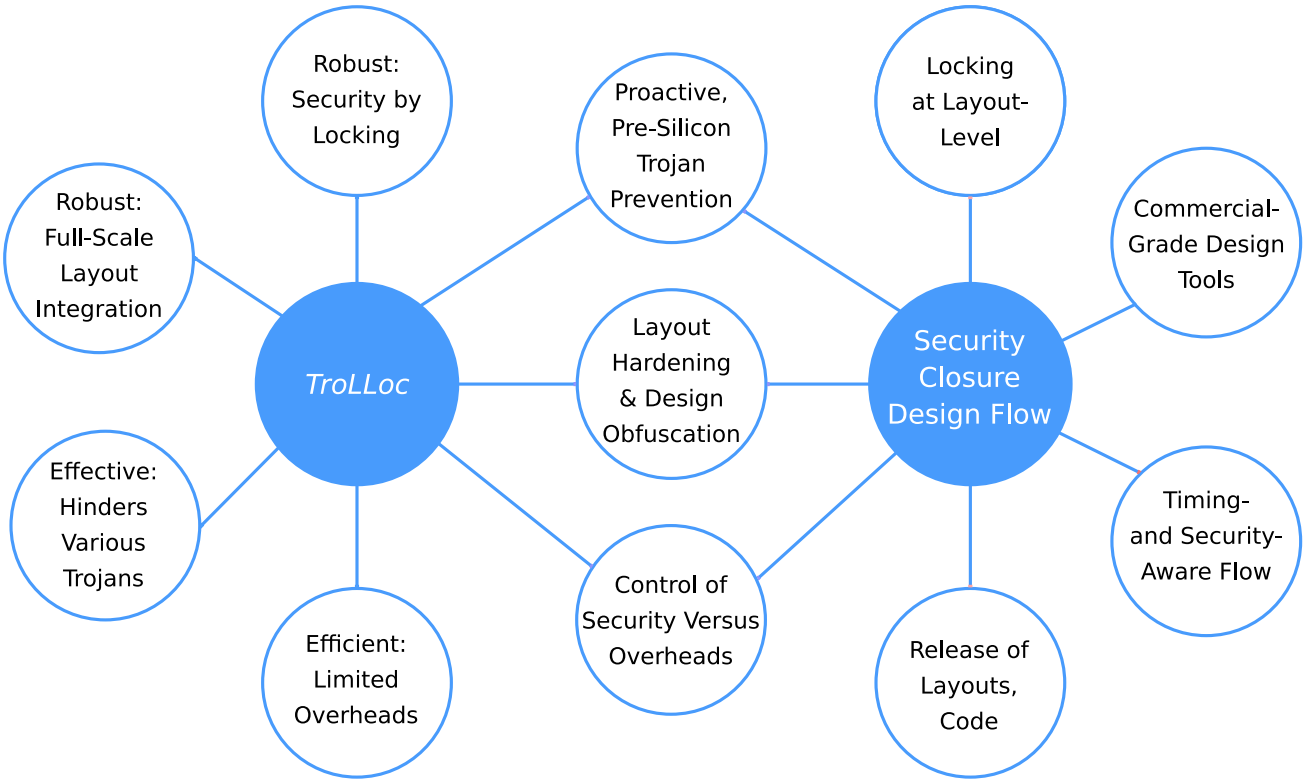


Fig. 1 High-level overview of our work. The two main contributions are the locking scheme, *TroLLoc*, and the design flow for IC security closure. Both are carefully orchestrated together to achieve pre-silicon Trojan mitigation.

2 Background and Motivation

2.1 Trojans

Real-World Relevance. ICs should provide a root-of-trust platform on which sensitive data operation can rely on. If that premise is broken, however, the confidentiality, integrity, and availability of any application is entirely at risk. For consumer electronics, Trojans that aim to compromise cryptographic circuitry, which exists in modern processors many years by now, represent such real-world risks [49]. For other domains like military or avionics, the stakes are even higher. Potential Trojan threats on military-grade ICs are discussed in [1, 59]. Remarkable case studies for Trojans in real ICs are described in [21, 46, 49, 50, 77].

Working Principles. Trojans are malicious hardware modifications [15, 76]. This concept is quite diverse, covering malicious modifications that may: (i) leak information from an IC, reduce its performance, or disrupt its working altogether; (ii) be always on, triggered internally, or triggered externally; (iii) be introduced through untrustworthy third-party IP, adversarial designers, mask generation, or manufacturing of ICs; etc. Besides, most Trojans require some additional layout-level resources like *open placement sites*, *free routing tracks*, and/or *timing slacks* [17, 38, 55, 62].[1]

Most if not all Trojans comprise a *trigger* and a *payload*; the trigger activates the payload on some attack condition, and the payload serves to perform an actual attack. Triggers are often based on low-controllability

[1] Modern IC layouts utilize *standard-cell libraries*, which are provided by foundries and their suppliers, along with the technology-dependent design rules in *technology libraries*. As indicated, the cells in those libraries are standardized, namely in their geometries, such that cells are relatively easy to place into regularly sized and arranged placement rows of an IC. Placement sites are bins of fixed width within such rows. Thus, open placement sites are bins that are not occupied yet by some standard cell. Further, an IC's interconnect comprises a stack of different metal layers; the latter are arranged into regular horizontal and vertical routing tracks. Thus, free routing tracks are stripes within these metal layers that are not utilized yet by some routing shape. Timing slacks can be thought of as performance margins. For an example in simple terms, assume that some part of an IC design, which does exhibit timing slacks, may become slower, e.g., due to insertion of additional Trojan logic. Then, the performance constraints of the overall design can still be met as long as the introduced slow-down is less than the available slacks.

nets (LCNs)[2]—to hinder their detection during testing, rendering them more stealthy—whereas payloads are targeting on sensitive assets like registers holding some key. Note that the trigger and payload components are implemented individually but are working together in tandem. For example, once some LCN is toggling, i.e., only for some rare and carefully crafted condition, the trigger component initiates the actual attack, e.g., the content of some sensitive registers is maliciously forwarded to non-privileged circuit parts or to primary outputs, or even more stealthily leaked through side-channels [49].

Threat Modeling. We assume post-design insertion of additive Trojans by adversaries residing within foundries. Among different scenarios [15, 76], this one is outstanding as powerful and versatile—related Trojans are not detectable at all during design-time and are also not limited to, e.g., denial-of-service attacks through removal or modification of existing logic, but can embed any malicious functionality at will.

An important novel concept for threat modeling in our work is introduced next. While similar ideas may have been used in the literature before, we are the first to explicitly formulate this concept as such. We differentiate between *first-order* versus *second-order* Trojan attacks. For the former, adversaries would seek to directly insert their Trojans into the IC layout under attack. For the latter, adversaries would first revise the layout to regain layout resources, to aid subsequent Trojan insertion. Both attacks can be supported by means of engineering change order (ECO) techniques [27, 41, 49, 73].[3] Besides, in case some layout defenses are put in place, conducting second-order attacks may require adversaries to bypass those as well. Depending on the nature of the defense, it may suffice to again utilize ECO techniques, e.g., to remove *fillers* and/or *spares*,[4] to rearrange the cell placement, etc. More advanced efforts may also be required; e.g., dedicated attacks would be needed to decipher and remove locking structures.

Countermeasures and Their Challenges. Countermeasures can be classified into:

1. Proactive, pre-silicon prevention (e.g., [7, 8, 16, 18, 19, 24, 28, 30, 32, 37, 44, 56, 57, 63, 68, 71, 72, 75]);
2. Reactive, post-silicon detection or monitoring (e.g., [2, 10, 19, 23, 29, 34, 67, 70, 74]);
3. Proactive detection, that is (a) pre-silicon inspection/verification (e.g., [12, 15, 20, 25, 42]), (b) post-silicon testing (e.g., [14, 15, 48]), or (c) post-silicon layout inspection (e.g., [50, 61, 66]).

There are various challenges for each of this class as follows. First, many post-silicon schemes rely on a "golden IC," i.e., a Trojan-free reference IC [2]. In threat models where the foundry is untrusted, this requirement is impractical. While designers may also conduct post-silicon verification via reverse engineering, this can be time-consuming, expensive, and difficult to scale across designs and technology nodes and even more so across IC batches. Second, both post-silicon monitoring and pre-silicon prevention require dedicated hardware features—if not secured properly, the related circuitry itself may be circumvented by adversaries during second-order attacks. Third, by construction, pre-silicon inspection/verification cannot tackle any Trojans inserted by the foundry—such Trojans do not exist yet at design-time. Fourth, post-silicon testing is challenging for multiple reasons: (i) regular testing procedures may not be helpful to determine some rarely triggered functional or parametric divergences introduced by Trojans; (ii) Trojans can be realized with largely varying logic structures and functional behaviour, making an identification of "common patterns" difficult [22]; while recent ML works like [26, 42] show promise toward that end, they also cannot achieve perfect accuracy; (iii) process variations as well as measurement noises make the observation of parametric effects introduced by Trojans more and more challenging for advanced IC technologies [33].

Given all these challenges, we argue that a *robust* scheme for proactive prevention is essential to try to hinder Trojans early on and as best as possible. We discuss the relevant prior art in more detail in Sec. 3.

2.2 Security Closure

Security closure seeks to proactively harden the physical layouts of ICs at design-time against various threats that are executed post design-time [17, 37, 38]. Security

[2] *Controllability* describes whether an end-user can establish specific signal assignments for some internal nets through the IC's primary inputs. Thus, LCNs are nets where end-users are challenged, across a wide range of input patterns, to achieve specific signal assignments and toggling activity.

[3] ECO techniques are an industry-wide standard for incremental changes of IC layouts, e.g., to revise the placement and routing of a small module within a larger IC design. Their (mis-)use for Trojan insertion has been successfully demonstrated only recently [17, 27, 41, 49, 73].

[4] Spares are redundant standard cells; they are placed early on but not fully connected yet. Such spares are helpful for last-minute ECO procedures, as their use would likely require only routing-level changes. Fillers are various non-functional cells that serve different purposes, e.g., acting as decoupling capacitors (decaps) to stabilize the IC's internal power supply network. Although spares and fillers can be used for pre-silicon prevention, by simply filling up all open placement sites with them, it is important to note that those components are easy to exploit for Trojan insertion as most of them can be readily removed without disrupting the core functionality of an IC [17, 50].

closure against Trojans means to control physical synthesis such that insertion of Trojan components is prevented altogether or at least becomes impractical, while managing the impact on design quality [17, 24, 30, 37, 38, 71, 72]. For example, aggressively dense layouts would leave only few open placement sites and few routing resources exploitable for Trojan insertion [17]. However, while such layouts are already challenging by themselves, in particular for managing design rule checks (DRCs),[5] such arguably naive approach may not be good enough for security closure. This is because, for one, advanced Trojans like *A2* [77] may require only 20 placement sites [62]; that few sites will remain even in any aggressively dense layouts, especially once filler and/or spare cells are considered as exploitable open sites as well. For another, recall that second-order attacks can regain layout resources before Trojan insertion, thereby rendering such implicit protection postulated by dense layouts ineffective in general.

Overall, we argue that efforts for security closure against Trojans need to address the challenges outlined in Sec. 1.

2.3 Logic Locking

Working Principles. As indicated, logic locking or locking for short, means to incorporate so-called key-gate structures that are controlled by a secret key [79]. Without the full knowledge of the key, logic locking ensures that the details of the design IP cannot be fully recovered and the IC remains non-functional. The key-gate structures are commonly realized using, e.g., X(N)OR gates [54], AND/OR gates [16], look-up tables (LUTs) [45], or multiplexers (MUXes) [51]. After manufacturing, preferably only after testing [78], the IC is activated: the secret key is loaded into a dedicated, on-chip tamper-proof memory by a trusted party.

While locking is largely known for protection of design IP [79], it can also serve for Trojan defense. In fact, different such schemes have been proposed: AND/OR locking [16], X(N)OR locking [44, 56], and custom locking [68].

[5] DRCs describe technology rules for manufacturability of ICs, in terms of cell placement (e.g., considering the so-called *N-well continuity* which is essential for IC semiconductors) and in terms of routing shapes (considering minimum areas, pitches, et cetera). Thus, any violation for DRCs can disqualify the whole IC from manufacturing. For modern ICs and their advanced technologies underlying, DRCs are becoming more and more complex [9].

Threat Modeling. Over the years, various attacks against the different implementations of locking have been proposed. Two different threat models are considered:

- The *oracle-guided* model assumes access to an activated IC, i.e., an IC with the correct key already loaded. Naturally, such asset is only available after manufacturing. Using another IC copy, the attacker has to extract the locked gate-level netlist through means of reverse engineering. The attacker then aims to decipher the key by analyzing the netlist, identifying meaningful input patterns through analytical means such as Boolean satisfiability (SAT) solving [60], applying those to the activated IC, and observing the responses to iteratively converge toward the correct key.
- The *oracle-less* model assumes only access to the locked gate-level netlist. The attacker aims to decipher the key by analyzing the structure and functionality of the netlist, e.g., by means of ML [5], identifying meaningful patterns and correlations of key-gates and correct key-bits.

Advanced Oracle-Less Attacks. Recently, ML-based attacks like *SAIL* [11], *SCOPE* [3], *OMLA* [6] and *MuxLink* [5], as well as structural attacks like *Resynthesis* [4], succeeded in identifying various relevant design features in locked circuits and in subsequently predicting key-gate structures. For example, using a graph representation of the IC design, *MuxLink* [5] learns on the structure of regular, unlocked parts of a design at hand that is specifically locked using MUX key-gates. After training, *MuxLink* seeks to decipher the input connections to MUX key-gates. More specifically, *MuxLink* considers each MUX key-gate's input at a time, connecting it to the output of the MUX—thereby bypassing the key-gate structure—and contrasts the resulting different outcomes to the trained knowledge to predict the more likely original structure, i.e., the circuitry before locking. For another example, *Resynthesis* [4] strengthens oracle-less attacks by generating many functionally equivalent but structurally diverse versions of the locked netlist. It does so by varying synthesis parameters, e.g., mapping effort and timing constraints, especially around key-gate-related paths.Each variant is then analyzed by an attack such as *SCOPE*, and the final inference is obtained via majority voting across variants, enabling a more holistic attack approach.

Given their high success rates, such oracle-less attacks represent a considerable threat for locking. Especially for locking-based, proactive Trojan prevention at design-time, such attack capabilities need to be taken in consideration. Otherwise, second-order attacks would remain a practical threat.

3 Prior Art and Their Limitations

Next, we discuss the prior art for proactive, pre-silicon Trojan prevention in more detail, including their limitations. We differentiate into locking-based versus layout-based schemes. A high-level comparison of prior art and ours is also outlined in Table 1. It is important to note that our work addresses *all* the limitations identified for prior art.

Locking-Based Trojan Prevention. Dupuis et al. [16] lock LCNs using AND/OR key-gates, to hinder insertion of Trojan triggers. They consider timing slacks, varying toggling thresholds, and balanced switching probabilities. Samimi et al. [56] follow similar principles as Dupuis et al., but utilize X(N)OR key-gates. Marcelli et al. [44] propose a multi-objective evolutionary algorithm, seeking to minimize LCNs and maximize the efficacy of X(N)OR locking at the same time. Šišejković et al. [68] secure inter-module signals against software-controlled hardware Trojans, using encryption circuitry along with regular locking. However, they do not specify/limit the type of key-gates.

The above prior art has limitations as follows. First, Dupuis et al. [16] cannot protect specific assets against Trojan insertion. This is because they utilize AND/OR key-gates to tune the controllability of selected nets, but not to obfuscate the design; see also the next point. Second, none show robustness against SOTA oracle-less attacks on locking. For example, Dupuis et al. [16] do not employ resynthesis after locking which, however, would be essential to obfuscate the correlation between key-gate structures and key-bit assignments [79]. Third, Šišejković et al. [68] can only protect against Trojans targeting on inter-module control signals, not on any of the modules' internal circuitry. Accordingly, Trojan attacks seeking to, e.g., manipulate certain computation steps, can still succeed. Fourth, none consider Trojans at the layout level; all are working at netlist level. This implies that none can conclusively prevent post-design insertion of Trojans in the real world. Finally, none explicitly prevent Trojan payloads. Šišejković et al. [68] are locking the design in general, without focus on triggers or payloads, and others are locking only LCNs to hinder the integration of Trojan triggers.

Layout-Based Trojan Prevention. Xiao et al. [75] propose to add built-in self-test components, arguing that tampering of those structures (e.g., by adversaries trying to regain layout resources for their Trojans) can be detected during post-silicon testing. Similarly, Ba et al. [7, 8] and Eslami et al. [19] insert other additional circuitry. Hossein-Talaee et al. [28] redistribute whitespace / open placement sites otherwise exploitable for Trojan insertion. Trippel et al. [63] employ specific routing structures, to prevent routing of Trojan components. Knechtel et al. [37], Guo et al. [24], Hsu et al. [30], Wei et al. [72], and Eslami et al. [18] all seek to harden layouts through physical synthesis. For example, Knechtel et al. propose techniques to locally increase both placement density (to hinder insertion of Trojans) and routing density (to hinder routing of Trojans), while Eslami et al. propose

Table 1 High-Level Comparison with Prior Art for Pre-Silicon Trojan Prevention

Works	Approach	Robust	Effective for: Funct./Layout	Efficient	Tested for Trojans	Artifacts Available
[16]	Locking	✗ [16, 56]	✗^{a} / ✗	✓✗	✗	✗
[56, 44]	Locking	✗✓ [6]	✗✓^{a} / ✗	✓✗	✗	✗
[68]	Locking	?	?^{a} / ✗	✓✗	✗	✗
[75, 7]	Additional Logic	✓✗ [49, 17]	✗ / ✓✗^{a}	✓✗	✗	✗
[8, 19]						
[28, 37]	Layout Hardening	✗✓ [49, 17]	✗ / ✗✓^{a}	✓✗	✗	✓✗
[24, 30]		✗✓ [49, 17]	✗ / ✗✓^{a}	✓✗	✗	✓✗
[18, 72]		✗✓ [49, 17]	✗ / ✗✓^{a}	✓✗	✗	✓✗
[63]	Routing	✓	✗ / ✓✗^{b}	✓	✓✗ [77]	✗
Ours: *TroLLoc*	**Locking & Layout Hardening**	✓	✓ / ✓	✓	✓ [40]	✓ [64]

Symbols – ✓ yes; ✗ no; ? unclear; ✓✗ yes but with some caveats; ✗✓ unlikely. Notes – *a*: lack/uncertainty of robustness is expected to undermine effectiveness; *b*: covers only routing resources

a holistic synthesis methodology; they demonstrate their work with an actual IC tape-out as well.

The above prior art has limitations as follows. First, none can truly protect specific assets against Trojans. In the absence of locking or other obfuscation schemes, the original design is fully accessible by adversaries. Second, except for Trippel et al. [63], none show robustness against second-order attacks. For example, for Hossein-Talaee et al. [28], shifting of whitespaces could be trivially reverted by ECO-assisted Trojan insertion [27, 49, 73]. While the works of [7, 8, 75] aim toward such robustness, the use of additional circuitry by itself does not resolve the fact that other parts of the IC design may remain exploitable. In particular, fillers and spares are utilized in most if not all prior art and, as indicated in Sec. 2.1, such components are easy to exploit. Third, for full efficacy, the work by Xiao et al. [75] requires 100% test coverage which can be very difficult to achieve for real-world, large-scale designs. Further, high utilization rates are challenging for their method. Both aspects are limiting the practicability of the scheme. Fourth, for the works in [7, 8, 75], the number of additionally required primary inputs scales with layout hardening. This is impractical; pads for primary inputs/outputs (PIs/POs) are large in actual ICs and, if not employed wisely, can considerably increase the chip outline, directly increasing cost for silicon area. Finally, except for Trippel et al. [63], none have evaluated the resilience of their proposed schemes against insertion of actual Trojans. Note that Trippel et al. consider only one specific Trojan, namely *A2* [77]; their evaluation cannot provide insights for any other kinds of Trojans.

4 Threat Modeling

Trojans. We base our assumptions on classical threat modeling for post-design insertion of additive Trojans [46, 49, 62, 63, 76]. Details are given next.

1. Adversaries reside within foundries, whereas the design process is trustworthy. Adversaries have full knowledge of the IC technology.
2. From 1) follows that adversaries have no knowledge of the original design, only of the physical layout at hand, which is protected through *TroLLoc*.
3. From 1) also follows that we do not account for Trojans introduced by, e.g., malicious third-party IP modules. Orthogonal countermeasures, like reactive monitoring (Sec. 2.1), may be applied against such threats as well, in conjunction to ours.
4. The adversary seeks to insert some Trojan(s), more specifically some dedicated trigger and payload components, into the physical layout. Toward that end, they may follow first-order attacks, i.e., direct insertion of Trojan components in the layout as is, or second-order attacks, i.e., revise the layout as much as necessary to insert the Trojan, as long as the layout then still meets DRCs and other final design checks.
5. We assume Trojans with triggers based on LCNs and payloads targeting on specific *assets*, i.e., security-critical components like registers holding some key. Also, Trojans are assumed to be implemented using regular standard cells.
6. We do not utilize any spares or fillers in our protected layouts. Thus, naive Trojan attacks exploiting such resources are ruled out.

Locking. Recall that we employ locking exclusively to hinder Trojan insertion. Thus, we apply oracle-less threat modeling [3–6, 11], in consistency with the above threat modeling for Trojans, as follows.

1. Adversaries reside within foundries. Thus, only oracle-less attacks are applicable.
2. From 1) follows that adversaries have no knowledge of the original design, only of the physical layout at hand, which is protected through *TroLLoc*.
3. Following Kerckhoffs' principle, all implementation details for *TroLLoc* are known to the adversaries; only the secret key-bits remain undisclosed. Adversaries also have full knowledge of the IC technology.
4. The adversaries' objective is to circumvent the locking scheme, to (a) understand the true functionality of the design, which is required for attacking specific assets, and (b) to regain layout resources, which is required for Trojan insertion in general.

5 Methodology

Recall that we seek to devise a scheme for proactive, pre-silicon Trojan prevention as an addition/alternative to only reactive, post-silicon Trojan detection. Also recall that related prior art all falls short in terms of robustness, effectiveness, and/or efficiency. Our approach—which can be summarized as locking and layout hardening applied

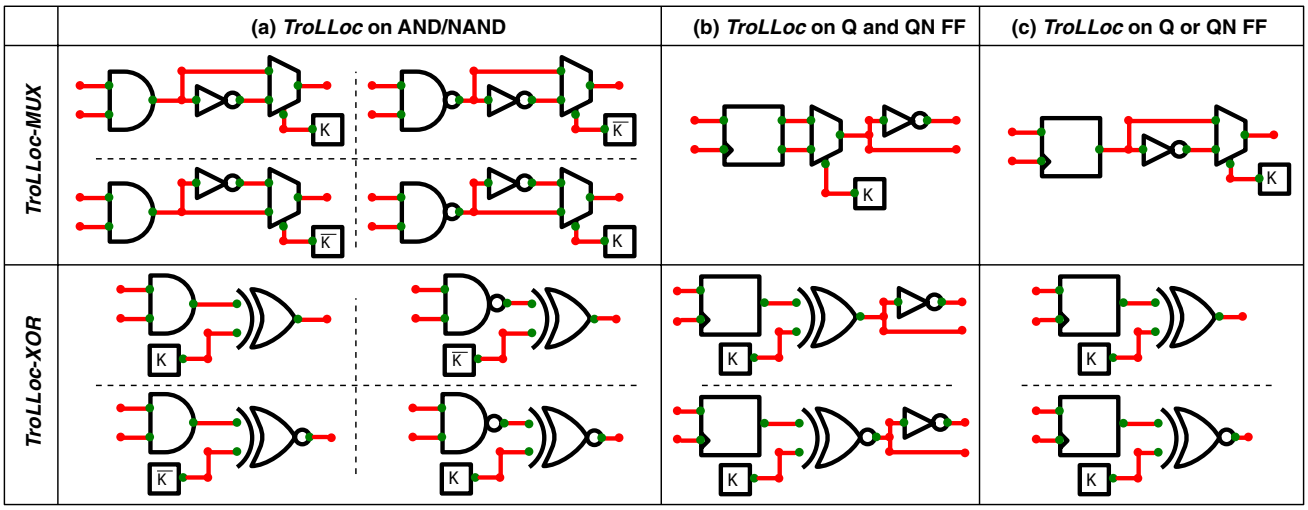


Fig. 2 Design of *TroLLoc* instances. For *TroLLoc-MUX* or *TroLLoc-XOR*, respectively, the key-bit is connected to the MUX select line or an XOR/XNOR input. (**a**) Locking of simple AND/NAND gates. Note that other simple gates are locked similarly, which is not illustrated here. When $K = 0$, all *TroLLoc* instances operate as AND gate; when $K = 1$, they function as NAND gate. Therefore, given any such *TroLLoc* instance, its functionality will remain unknown without the correct key-bit. (**b, c**) Locking of different types of FFs. For (**b**), the key-bit dictates which output signal holds Q and which QN; for (**c**), the key-bit dictates whether Q or QN is put out. For *TroLLoc-XOR*, we randomly pick only one of the FF's outputs and randomly pick XOR or XNOR as key-gates.

carefully in unison—aims to hinder both first-/second-order attacks of post-design Trojan insertion.

For locking, we devise a scheme which is (i) resilient against various SOTA attacks and (ii) directly integrated at layout-level. For layout hardening, unlike prior art, we neither employ trivial filler/spare cells nor any other vulnerable circuitry but only locking instances. To protect IC layouts holistically, our methodology carefully embeds, in a security-aware manner, as many locking instances during physical synthesis as practically possible, i.e., while keeping the design quality well under control.

Next, we propose the locking scheme, *TroLLoc*. Thereafter, we devise its integration into a security-and-design-aware synthesis flow based on commercial-grade IC tooling.

5.1 *TroLLoc* Locking Scheme

Approach. Note that both MUX and X(N)OR gates are reasonable choices for key-gates in locking schemes. In fact, both types are used extensively in prior art, with specific attacks being tailored for each type (Sec. 2.3). Thus, we consider both types for implementation as well as all the experimental investigation. For brevity, we will refer to "*TroLLoc* implemented with MUXes as key-gates" as *TroLLoc-MUX* and "*TroLLoc* implemented with X(N)ORs as key-gates" as *TroLLoc-XOR* in the remainder of this paper.

Unlike prior art for locking-based Trojan prevention (Sec. 3), *TroLLoc* utilizes key-gates in a similar way as done in SOTA locking schemes, with the specific intent to avoid information leakage arising from the key-gates' structure and connectivity. *TroLLoc* employs simple but robust key-gate structures with localized connectivity, described next.

Learning-Resilient Key-Gate Structures. *TroLLoc* ensures that key-bits can be fully randomized, without inducing correlations to the locked gate's true functionality. Thus, *TroLLoc* shall be robust against ML-based attacks by construction. To do so, *TroLLoc* instances render any locked gate interchangeable with their complementary counterparts, e.g., a locked NAND gate can act as AND or NAND, only depending on the key-bit. More specifically, one picks randomly from the different possible configurations applicable for *TroLLoc-MUX* or *TroLLoc-XOR*, as shown in Fig. 2.

Continuing the example of AND/NAND gates, for *TroLLoc-MUX*, one would consider the four configurations in the upper part of Fig. 2(a). Note how these configurations are pairwise indistinguishable. Also note how the two configurations to the right are based on transforming the original gates to their counterparts—this serves for further obfuscation of the overall layout regarding the distribution of gate types. For *TroLLoc-XOR*, aside the random transformation of original gates, we also randomly select XOR/XNOR key-gates.

Concerning flip-flops (FFs), these can be locked as is, by connecting the Q and/or QN ports[6] to a randomly selected, appropriate key-gate structure shown in Fig. 2(b, c).

No Further Information Leakage from Physical Layouts. Aside from the fact that *TroLLoc* key-gate structures are pairwise complementary, which applies to most if not all locking schemes, there are *no* direct correlations, neither between the types of original versus locked gates, nor between the locked gates and the key-bits.

For *TroLLoc-MUX*, all internal nets related to the MUX key-gate share a common timing path and are jointly optimized by the synthesis engine. Thus, an attacker seeking to extract any additional information from the underlying timing paths, driver strengths, etc., cannot succeed. For *TroLLoc-XOR*, this fact applies similarly as well.

[6] For circuit-level efficiency, standard-cell libraries typically contain two types of FFs: those with the stored data provided as complementary pair of output signals, called Q and QN, and those with only one output signal, either for the original data Q or the inverted data QN. Design tools then automatically instantiate, for each FF, the type which is most suitable/efficient.

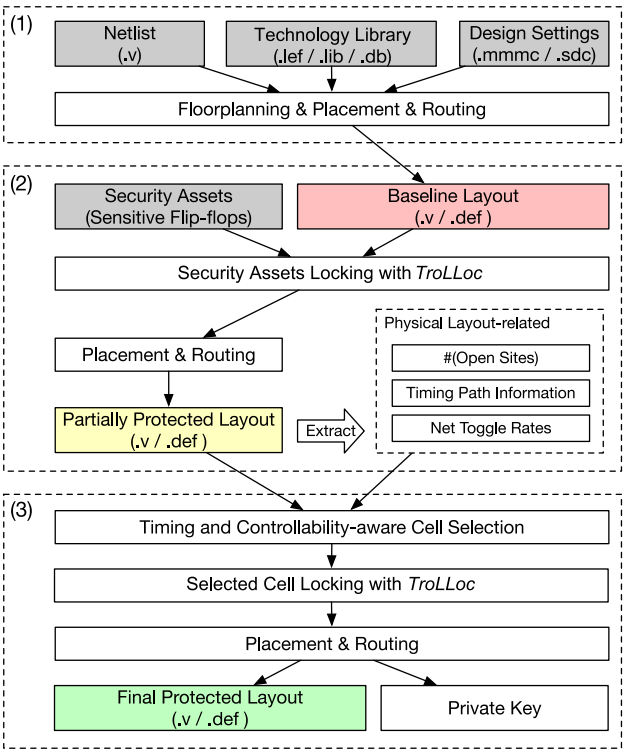


Fig. 3 Our physical-synthesis flow. (**1**) Initial synthesis of baseline layouts. (**2**) Locking of security assets, followed by synthesis. (**3**) Locking of further components, considering timing and controllability, followed by synthesis.

5.2 Security-and-Design-Aware Physical Synthesis

Next, we introduce our physical-synthesis flow that is capable of hardening any post-route layout with *TroLLoc* instances, incurring well-controlled timing and area overheads in the process. As outlined in Fig. 3, we start with the original netlist and other necessary data to generate a baseline layout and then carefully interleave locking and physical synthesis throughout two stages to produce a final protected layout.

Two-Stage Scheme. In the first stage, we initially lock all security-critical FFs, i.e., FFs defined as assets by the user, with *TroLLoc* instances. After locking, we conduct placement and routing (P&R), and obtain a partially protected layout (PPL). Note that a PPL has only assets protected; no other parts like LCNs are protected yet. Further, depending on the number of user-provided assets, a PPL can still exhibit a relatively low utilization, potentially leaving the layout vulnerable to Trojan insertion.

In the second stage, we aim at locking as many more components as practically possible, while prioritizing the locking of LCNs, all without degrading the design quality. Toward that end, we start by extracting some specifics obtained from the PPL: remaining open sites, timing paths, and controllability of all nets. Then, we derive the number of cells to be locked considering open sites and timing; this step is detailed below in the paragraph *On-Demand Key Length*. Next, we perform cell selection considering timing and controllability; this step is detailed below in the paragraph *Timing-and-Controllability-Aware Cell Selection*. Then, all selected cells are locked. After that, P&R is conducted again, and we retrieve a highly-utilized, final protected layout.

Note that initial experiments had shown that this two-stage approach is more feasible and efficient, as in more timing-friendly, over a single-stage approach. That is, when we did directly use the specifics extracted from the baseline layouts (marked in red in Fig. 3) for cell selection and locking, it was much harder to achieve timing closure, due to an accumulation of slack estimation errors.

Storage of Key-Bits. For each *TroLLoc* instance, there will be one corresponding key-bit, requiring some facility to store all the key-bits. We employ a large shift register for that, without loss of generality but for the following reasons:

– Prior art often considers adding a dedicated PI for each key-bit, which is not scalable. For our, only two additional PIs are required (data in, load) for any key length.

– Such shift register incurs negligible impact on performance and power. For an IC locked with k key-bits using *TroLLoc*, loading the key will take k clock cycles. This is done once during the initial boot-up, while the main circuitry is still hold in reset. Once the load signal is reset to `low`, the key-bits will remain stable and all related FFs are only consuming some static power.
– The related FFs are also helpful for locally filling/hardening open placement sites, whereas bulky memory blocks would make this task rather complicated.

On-Demand Key Length. *TroLLoc* instances help to occupy open placement sites with their INVs, MUXes/X(N) ORs, and FFs. For *TroLLoc-MUX* for example, we determine the key length for the second stage as follows:

$$k = floor\left(\frac{num_open_sites}{size(INV) + size(MUX) + size(FF) + \alpha}\right), \quad (1)$$

where $size(INV)$ represents the size of the smallest INV cell in the library, etc., and α is a parameter for timing budget. For *TroLLoc-XOR*, we determine the key length similarly. Note that commercial IC tools conduct timing optimization by gate sizing, repeater insertion, etc. Thus, we need to reserve some space for such optimization efforts during locking. Accordingly, for designs where timing closure is more challenging, we will utilize a larger α value, and vice versa.

Timing-and-Controllability-Aware Cell Selection. Since locking more and more *TroLLoc* instances will introduce further delays to all related timing paths, the selection of cells to lock becomes critical for timing closure. Thus, we propose a scoring function $cellScore(c, N, P)$ to comprehensively describe the priority of a cell c as follows:

$$cellScore(c, N, P) = \sum_{n \in N(c)} netScore(n, P), \quad (2)$$

$$netScore(n, P) = \frac{1}{1 + exp(-2 \cdot MS(n, P))} \cdot \frac{1}{TPC(n) + 10^{-3}}, \quad (3)$$

$$MS(n, P) = \begin{cases} \min\limits_{p \in P(n)} p.slack & , \text{if } |P(n)| > 0, \\ -0.5 & , \text{otherwise}, \end{cases} \quad (4)$$

with terms described in Table 2. More details are given next.

First, the score is represented as sum of net-level scores to generalize to multi-output cases, e.g., both Q and QN of a FF are used. Second, the net-level score is devised to jointly consider timing and controllability. By using a sigmoid function, the net-level score value remains positive even for negative but small slacks. Third, since $TPC(n) \in [0, 2]$ and nets with $TPC \leq 0.1$ are considered as LCNs in this work, we add a small margin (10^{-3}) to avoid both `div-by-0` and $TPC(n)$ from dominating the score. Fourth, when calculating MS, some nets may not be covered by any of the reported timing paths—this scenario in general is a well-known limitation of commercial IC tools. For those nets, $MS(n, P)$ returns -0.5 as a "fall-back value" which is close to observed average of negative slacks. Thus, this value serves to conservatively penalize cells with unknown timing.

Table 2 Notations for Cell Selection

Term	Description
C	The set of standard cell instances
$\boldsymbol{c}$	**A cell instance from C**
N	The set of nets
$\boldsymbol{n}$	**A net from N**
$N(c)$	The set of nets driven by c
$\boldsymbol{P}$	**The set of timing paths**
$P(n)$	The set of timing paths covering n
$\boldsymbol{MS(n, P)}$	**The minimum slack of paths covering n**
$TPC(n)$	The average nr. of toggles per clock cycle for n

Algorithm 1 Cell Selection Considering Timing and Controllability

Require: Standard Cells C, Nets N, Timing Paths P, Key Length K, Locking Delay σ.
Ensure: The Set of Cells to Lock C'.
1: $C' \leftarrow \{\}$;
2: **while** $|C'| < K$ **do**
3: **for all** cell $c \in C$ **do**
4: $c.score \leftarrow cellScore(c, N, P)$; ▷ Score calculation for each cell
5: Let c_h be the cell with the highest score in C;
6: $C'.append(c_h)$;
7: $C.remove(c_h)$;
8: **for all** net $n \in N(c_h)$ **do**
9: **for all** timing path $p \in P(n)$ **do**
10: $p.slack \leftarrow p.slack - \sigma$; ▷ Pessimistic slack estimation
11: **return** C';

We propose an iterative cell-selection procedure in Algorithm 1. There, in each iteration, we calculate the cell scores (line 3–4) and pick the cell(s) with the highest score (line 5–7). Next, we update the recorded slacks of all affected timing paths based on σ, i.e., a pessimistic estimate of delays introduced by *TroLLoc* instances. For *TroLLoc-MUX* (or *TroLLoc-XOR*), σ is defined as the sum of worst-case delays for `INV_X1` and `MUX2_X1` (or `XOR2_X1`), which are the default options for actual cells for *TroLLoc* instances. Note that the worst-case delays are derived for the matching corners (line 8–10).[7]

[7] *Corners* describe different sets of parameters for characterizing an IC's operation and performance under varying ambient conditions.

6 Experimental Investigation

Scope. We conduct a series of case studies as follows. In Sec. 6.1, we pursue security closure in general, showing that *TroLLoc* can harden layouts against Trojan insertion with reasonable overheads. In Sec. 6.2, we perform actual layout-level insertion of various Trojans, showing that *TroLLoc* is effective and robust against such real-world attacks, which were largely overlooked in prior art. In Sec. 6.3 and Sec. 6.4, respectively, we show the resilience of *TroLLoc-MUX* and *TroLLoc-XOR* against second-order attacks where advanced adversaries would try to bypass the locking defenses before Trojan insertion.

Tools and Implementation. As indicated, we base our work on a commercial-grade design flow. For experiments on *TroLLoc-MUX* and *TroLLoc-XOR*, respectively, *Innovus 20.14* and *Innovus 21.14* (by *Cadence*) are used for physical synthesis.[8] The methodology is implemented in custom *TCL* scripts and *Python* code.

Benchmarks and Technology Setup. We employ the ISPD'22 benchmark suite [38]. The suite comprises layouts for a range of crypto cores as well as the *openMSP430* microcontroller, all implemented with the well-known and publicly available *Nangate 45nm Open Cell Library* [47]. The suite also describes security assets. Since the suite was originally synthesized by some legacy versions, namely *Innovus 18.13* and *Innovus 16.15*, we initially resynthesize all designs at our end. We do so for floorplan utilization rates similar to the original baseline layouts; only the rates for *AES_3*, *openMSP430_2*, and *TDEA* are set 10% lower, as needed to lock all security assets.

6.1 Case Study 1: Security Closure on ISPD'22 Benchmarks

Setup and Metrics. We follow the general setup outlined above. For metrics, we focus on layout-related metrics that reflect on the trade-offs for security closure and design quality. More specifically, we consider (i) utilization, open placement sites, and track utilization for security closure and (ii) power, performance, and DRC and other checks for design quality. Note that these metrics are established for security closure [38, 41, 62].

Results. We present security and layout results for the resynthesized baseline layouts versus the final, protected layouts in Table 3. In Fig. 4, we provide an example for *TroLLoc-MUX* applied on the *Camellia* benchmark. More details are discussed next.

First, all the protected layouts exhibit much higher utilization rates: 93.6–99.0%.[9] Recall that, unlike prior art, we do not utilize any fillers or spares toward that end. Thus, this remarkable achievement helps to significantly reduce open placement sites, namely by 90.3% and 89.5% on average, respectively, for *TroLLoc-MUX* and *TroLLoc-XOR*. In turn, this renders all layouts more resilient against Trojan insertion—the less open sites remain, the more difficult Trojan insertion becomes [38, 41, 62]. We also achieve higher routing track utilization (defined as *total routed wire length / total track length*); thus, routing for Trojans will become more challenging as well [38, 41, 62]. Second, despite the extremely high utilization achieved without using any filler or spares, all protected layouts are without DRC violations. This proves the efficiency and real-world relevance of our proposed scheme, in particular of the design flow for security closure.

For design overheads induced by *TroLLoc* during security closure, we note the following. On average, total power is increased by 18.5% for *TroLLoc-MUX* and 17.7% for *TroLLoc-XOR*, respectively. This is primarily due to all the additional cells introduced through *TroLLoc* instances. As expected, designs with low initial utilization (e.g., *CAST* and *MISTY*) tend to induce a relatively high power increase, whereas designs with more reasonable initial utilization (e.g., *AES_1* and *openMSP430_2*) induce much more limited power increases. For performance, the average and the maximal worst negative slacks (WNS)[10] are 1.41% and 3.10% for *TroLLoc-MUX*, and 0.54% and 1.40% for *TroLLoc-XOR*, respectively. This is reasonable as well, given all the additional *TroLLoc* circuitry. Also note that, unlike DRCs or other critical design checks, WNS violations do *not* render the IC unfit, neither for manufacturing nor for actual use; the only requirement for such ICs would be to operate them with an accordingly reduced clock period.

Finally, the following **research question** arises: *facing such remarkable layout hardening, can adversaries still*

[8] The use of different versions is merely due to the fact that these two different sets of experiments were conducted at different points in time. We confirmed that only negligible differences in design quality arise, e.g., differences in utilization are within 1%; thus, these two versions of *TroLLoc* remain compatible and results as well as findings are comparable.

[9] In general, achievable utilization is subject to the design at hand and also to the technology library. For our work, showcasing such ultra-high utilization is important—it supports our claim of truly hardening layouts against Trojan insertion. As indicted, prior art like [75] is challenged by high utilization.

[10] WNS quantifies the worst timing path, i.e., the path with the largest negative value and, thus, the path that exhibits the largest timing violation.

Table 3 Layout and Security Results on the ISPD'22 Contest Benchmark Suite

	Design	Baseline Layout (Resynthesized)						Protected Layout (Final)							
		Utils	*#OS*	*TU*	*WNS*	*TNS*	*Power*	*Utils*	*#OS*	*ΔOS*	*TU*	*WNS*	*TNS*	*Power*	*KL*
	AES_1	75.4%	43,980	8.7%	0.000	0.000	59.957	96.2%	6,838	-84.5%	11.1%	-0.013	-0.043	60.552	1,199
	AES_2	**75.1%**	**44,420**	**8.7%**	**-0.001**	**-0.003**	**59.441**	**96.3%**	**6,649**	**-85.0%**	**10.9%**	**-0.008**	**-0.017**	**61.040**	**1,202**
	AES_3	85.7%	22,129	9.7%	-0.001	-0.002	59.869	96.6%	5,225	-76.4%	11.2%	-0.031	-4.657	62.787	441
	Camellia	**49.5%**	**33,919**	**9.5%**	**1.194**	**0.000**	**1.233**	**98.9%**	**753**	**-97.8%**	**13.3%**	**0.015**	**0.000**	**1.488**	**1,271**
	CAST	49.1%	54,444	9.1%	0.047	0.000	3.136	93.6%	6,879	-87.4%	12.7%	-0.134	-1.455	3.912	1,572
	MISTY	**48.5%**	**43,359**	**8.6%**	**-0.021**	**-0.037**	**2.238**	**94.4%**	**4,686**	**-89.2%**	**12.4%**	**-0.140**	**-1.515**	**2.966**	**1,215**
	openMSP430_1	49.7%	32,799	6.2%	0.000	0.000	0.375	97.9%	1,389	-95.8%	12.6%	0.000	0.000	0.544	1,218
	openMSP430_2	**70.5%**	**15,125**	**7.7%**	**0.000**	**0.000**	**1.239**	**97.1%**	**1,497**	**-90.1%**	**10.6%**	**-0.006**	**-0.015**	**1.369**	**543**
	PRESENT	49.8%	6,284	4.4%	6.694	0.000	0.198	98.0%	245	-96.1%	6.8%	4.935	0.000	0.235	241
	SEED	**49.1%**	**54,444**	**9.1%**	**0.047**	**0.000**	**3.136**	**94.3%**	**6,056**	**-88.9%**	**12.7%**	**-0.182**	**-1.072**	**3.908**	**1,612**
	SPARX	49.9%	69,979	6.3%	2.452	0.000	2.164	98.8%	1,658	-97.6%	14.0%	0.027	0.000	2.822	2,582
TroLLoc-MUX	**TDEA**	**70.4%**	**5,559**	**6.8%**	**0.049**	**0.000**	**1.084**	**98.4%**	**304**	**-94.5%**	**8.0%**	**0.027**	**0.000**	**1.153**	**214**
	AES_1	74.9%	44,781	8.5%	0.000	-0.001	58.671	96.1%	6,922	-84.5%	10.7%	-0.014	-0.160	59.689	1,200
	AES_2	**74.7%**	**45,152**	**8.5%**	**0.000**	**0.000**	**58.766**	**95.0%**	**8,971**	**-80.1%**	**10.6%**	**-0.007**	**-0.057**	**59.554**	**1,149**
	AES_3	86.3%	21,244	9.8%	-0.004	-0.011	59.727	95.5%	6,921	-67.4%	10.7%	-0.003	-0.042	60.719	416
	Camellia	**49.5%**	**33,930**	**9.2%**	**0.856**	**0.000**	**1.237**	**98.7%**	**864**	**-97.5%**	**12.2%**	**0.075**	**0.000**	**1.478**	**1,348**
	CAST	49.1%	54,501	8.9%	0.023	0.000	3.143	94.6%	5,774	-89.4%	11.7%	-0.094	-0.934	3.867	1,804
	MISTY	**48.5%**	**43,365**	**8.7%**	**-0.013**	**-0.013**	**2.266**	**93.9%**	**5,102**	**-88.2%**	**11.8%**	**-0.097**	**-1.554**	**2.945**	**1,315**
	openMSP430_1	49.6%	32,813	6.1%	0.000	0.000	0.375	98.7%	875	-97.3%	10.1%	0.000	0.000	0.542	1,337
	openMSP430_2	**70.4%**	**15,170**	**7.7%**	**-0.004**	**-0.008**	**1.238**	**97.4%**	**1,349**	**-91.1%**	**10.0%**	**0.000**	**0.000**	**1.371**	**582**
	PRESENT	49.8%	6,274	4.2%	5.654	0.000	0.197	97.4%	327	-94.8%	6.2%	4.994	0.000	0.234	255
	SEED	**49.1%**	**54,501**	**8.9%**	**0.023**	**0.000**	**3.143**	**94.1%**	**6,364**	**-88.3%**	**11.6%**	**-0.024**	**-0.106**	**3.745**	**1,802**
	SPARX	49.8%	70,038	6.2%	2.389	0.000	2.161	99.0%	1,419	-98.0%	13.3%	0.319	0.000	2.888	2,746
TroLLoc-XOR	**TDEA**	**70.0%**	**5,634**	**6.8%**	**0.021**	**0.000**	**1.083**	**99.0%**	**185**	**-96.7%**	**8.1%**	**0.017**	**0.000**	**1.173**	**226**

Utils: utilization after physical synthesis; #OS: nr. of open sites; TU: track utilization; WNS (ns): worst negative slack; TNS (ns): total negative slack; Power (mW): total power; Δ OS: reduction of nr. of open sites; KL: key length (bits), accounting both for locked assets and other locked cell instances.

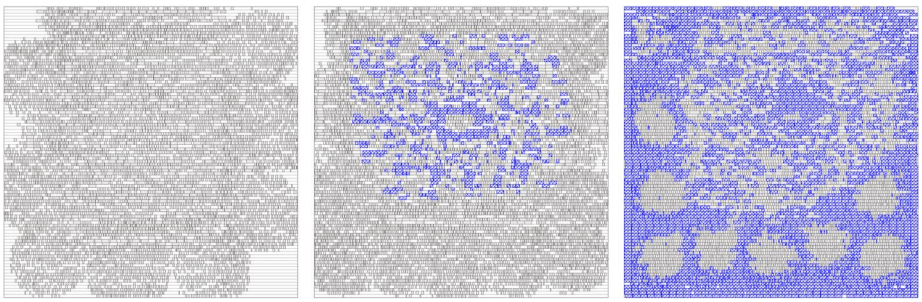


Fig. 4 Layout illustrations for the benchmark *Camellia* under *TroLLoc-MUX*. (**Left**) baseline layout, i.e., after initial resynthesis; (**middle**) after first stage, i.e., locking of security assets; (**right**) final protected layout, i.e., after second stage, locking of other cells considering timing and controllability. Cells introduced by *TroLLoc-MUX* instances are marked in blue, whereas others are marked in grey. The utilization increases from 49.5% (left) over 59.2% (middle) to finally 98.9% (right); see also Footnote 9.

succeed for Trojan insertion, or will they be hindered, e.g., by DRC violations and/or other real-world design challenges? To answer this, we conduct actual Trojan insertion on the hardened layouts next, in Sec. 6.2.

6.2 Case Study 2: Actual Trojan Insertion with ISPD'23 Suite

Setup For Trojans and Trojan Insertion. We employ the ISPD'23 suite [17, 40] in parts, namely for their Trojans and ECO techniques for Trojan insertion. However, for fair comparison, we refrain from using the ISPD'23 suite in full. That is, to put results for Trojan insertion into the context of the prior security and layout analysis in Sec. 6.1, we continue with the ISPD'22 benchmarks here. Note that only a subset of Trojans are readily compatible with the ISPD'22 benchmarks, namely those targeting on the very same cell instances. We obtain all 9 related Trojans by performing technology mapping, without any optimization, from the *ASAP7* technology library [13, 65] used for the ISPD'23 contest to the *Nangate 45nm* library used for the ISPD'22 benchmarks.

Metrics. Based on the number of DRC and/or other design-check violations, and the total negative slacks (TNS),[11] we derive a conservative interpretation of the final result, i.e., the best-case outcome with least violations and smallest TNS value across all three different modes for Trojan insertion as in the ISPD'23 contest. The threshold between 'manageable' versus 'excessive' violations are, following some rules of thumb from the real-world of IC design, set to > 25 DRC violations and TNS values exceeding 20% of the clock period. For cases where both the baseline and the protected layouts lead to the same final result, we further compare the degree of violations, to judge whether *TroLLoc* managed to incur more or less challenges for the adversary toward final design closure after Trojan insertion. Here, DRC violations are prioritized over TNS violations. Doing so is crucial as most, if not all, DRC violations must be fixed by adversary in order to release the design for actual manufacturing without raising suspicions or even failures, whereas *some* TNS violations may be kept as these would 'merely' force the IC to operate correctly only for a reduced clock period (such outcome may well be blamed on process variations).

Results. We present the outcomes for Trojan insertion in Table 4, allowing us to assess the effectiveness of *TroLLoc* versus the practicality of threats arising from layout-level Trojan insertion. Across the 9 Trojans in total, insertion was made more difficult in 7 case by *TroLLoc-MUX* and in 8 cases by *TroLLoc-XOR*, respectively. In 4 and 6 out of those cases, respectively, Trojan insertion was rendered even entirely impractical, as excessive violations were induced on the attackers' efforts. More details are discussed next.

First, *TroLLoc* is particularly effective for *Camellia*, i.e., a benchmark where ultra-high utilization is achieved for the protected layouts. Related layout examples are shown in Fig. 5: while Trojan insertion was successful without any violations for the baseline layout, it failed due to excessive violations for the protected layout. For the latter, also compare to Fig. 4, and note how the Trojan logic just barely fitted into the few remaining open sites. Due to the resulting spread-out nature of the Trojan, excessive timing violations occurred. Further, due to the ultra-high utilization, a large

[11] Here, we focus on TNS instead of WNS. TNS serves well to quantify the challenge for adversaries to fix all the timing issues, whereas WNS only quantifies by how much the whole IC violates timing.

Table 4 Results for ISPD'23 Trojan Insertion

	Design – Trojan	Baseline Layout (Resynthesized) Md 'reg' #Vs	TNS	Md 'adv' #Vs	TNS	Md 'adv2' #Vs	TNS	Res	Protected Layout (Final) Md 'reg' #Vs	TNS	Md 'adv' #Vs	TNS	Md 'adv2' #Vs	TNS	Res
TroLLoc-MUX	Camellia – burn rnd	0	0.000	0	0.000	0	0.000	✗	11	0.000	21	0.000	89	-3.022	✗✓
	Camellia – burn targ	0	0.000	0	0.000	0	0.000	✗	11	0.000	21	0.000	89	-3.022	✗✓
	Camellia – fault rnd	28	0.000	0	0.000	0	0.000	✗	95	0.000	33	0.000	26	-0.189	✓✗
	Camellia – fault targ	19	0.000	0	0.000	0	0.000	✗	54	-0.102	28	0.000	51	-2.611	✓✗
	Camellia – leak rnd	0	0.000	0	0.000	0	0.000	✗	62	0.000	61	0.000	2,737	-99.210	✓✗
	Camellia – leak targ	0	0.000	0	0.000	0	0.000	✗	62	0.000	61	0.000	2,737	-99.210	✓✗
	SEED – burn rnd	0	-3.265	0	-3.293	0	-3.184	✓✗	0	-2.249	0	-2.224	40	-2.395	✓✗ ⊖
	SEED – fault rnd	12	-3.633	0	-3.624	0	-3.353	✓✗	35	-2.168	18	-2.326	24	-2.045	✓✗ ⊕
	SEED – leak rnd	0	-3.177	0	-3.188	0	-3.290	✓✗	0	-2.137	0	-2.199	11	-4.370	✓✗ ⊖
TroLLoc-XOR	Camellia – burn rnd	0	0.000	0	0.000	0	0.000	✗					146	-0.222	✓✗
	Camellia – burn targ	0	0.000	0	0.000	0	0.000	✗					146	-0.222	✓✗
	Camellia – fault rnd	40	0.000	0	0.000	0	0.000	✗					164	-0.769	✓✗
	Camellia – fault targ	20	0.000	0	0.000	0	0.000	✗					95	-0.002	✓✗
	Camellia – leak rnd	0	0.000	0	0.000	0	0.000	✗					300	-2.531	✓✗
	Camellia – leak targ	0	0.000	0	0.000	0	0.000	✗					300	-2.531	✓✗
	SEED – burn rnd	0	-1.188	0	-1.312	0	-1.170	✓✓	0	-0.750	3	-0.798	10	-7.123	✓✓ ⊖
	SEED – fault rnd	14	-1.683	0	-1.543	0	-0.938	✓✓	25	-0.728	6	-0.887	10	-1.790	✓✓ ⊕
	SEED – leak rnd	0	-1.301	0	-1.163	0	-1.430	✓✓	16	-0.770	13	-0.802	10	-3.100	✓✓ ⊕

Md: Trojan insertion mode as in ISPD'23 contest; #Vs: nr. of DRC and/or other design-check violations; TNS (ns): total negative slack; 👍 Trojan insertion failed, due to exceedingly high utilization after insertion. Res: conservative interpretation of final result, i.e., best outcome for adversaries across all three modes, labelled from defender's perspective as follows – ✗ at best, Trojan insertion succeeded without any violations; ✓✗ at best, Trojan insertion succeeded with manageable violations (i.e., *TroLLoc* failed partially); ✓✓ at best, Trojan insertion succeeded, albeit with excessive violations (i.e., *TroLLoc* mainly successful). Note that, for cases where the final results' interpretation is the same across the baseline and protected layouts, we further report whether violations have ⊕ increased or ⊖ decreased for Trojan insertion into the protected layout over Trojan insertion into the baseline layout.



Fig. 5 Layout illustrations for the benchmark *Camellia*, after Trojan *leak targ* is inserted using the 'reg' mode. (**Left**) Trojan inserted into the baseline layout; (**right**) Trojan inserted into the final layout protected by *TroLLoc-MUX*. All benign cells, including *TroLLoc-MUX* instances, are marked in grey, whereas Trojan cells are marked in red.

number of DRC violations occured as well. Second, *TroLLoc-XOR* is slightly more effective than *TroLLoc-MUX*. Interestingly, the layouts protected by *TroLLoc-XOR* exhibit slightly lower utilization numbers (Table 3). This implies that layout hardening against post-design insertion of actual Trojans is more intricate than 'only' requiring to push for ultra-high utilization. This finding argues the converse of earlier prior art like [8, 37, 75], but aligns with more recent and more practical works like [41, 73]. Third, although *TroLLoc* cannot prevent Trojan insertion altogether, it is still effective. Here it is important to note that, while not directly comparable due to different technology setups, none of the winning teams of the ISPD'23 contest could fully prevent Trojan insertion either [17, 41]. This implies that post-design insertion of actual Trojans, specifically through commercial-grade ECO techniques, represents a severe threat. This finding also aligns with recent prior art [27, 49, 73]. Fourth, while not explicitly quantified here, note that, even in cases where Trojan insertion succeeded with manageable violations, the adversaries would still face two challenges: (i) resolve the violations, which becomes ever-more challenging for these ultra-dense layouts, and (ii) utilize the Trojan later on in the field. For (i), recall that adversaries would have to tackle most/all DRC violations and also keep TNS violations within tight bounds. For (ii), all security assets targeted at by Trojans are locked and thus obfuscated, i.e., all the assets' signal values could be either as is or inverted, depending on the unknown locking

key. Thus, while not explicitly quantified either, the actual Trojan implementations would become more complex and, thus, more difficult to insert.

To summarize and answer the research question raised in Sec. 6.1: adversaries are considerably but not entirely hindered by *TroLLoc* for post-design Trojan insertion, whereas an assessment of actual prospects requires case-by-case studies. This aligns well with the findings from the ISPD'23 contest [17, 41].

6.3 Case Study 3: Second-Order Attacks on *TroLLoc-MUX*

Setup for *MuxLink***.** We obtain the *MuxLink* implementation from [5] and revise it as follows. First, we extend the one-hot feature vectors of *MuxLink* to capture all possible gates utilized in the ISPD'22 suite. Second, consistent with the original operation, we represent each gate, PI, and PO as a separate node in a graph, with all connections between gates and ports represented as edges. Note that the connections between the input/output of *TroLLoc-MUX* key-gates are kept as test set for prediction, while all others are used as training set. Finally, we adopt the same graph neural-network configuration and training hyperparameters as in [5]; specifically, the number of hops while extracting subgraphs is set to 3 and the threshold for post-processing is set to 0.01.

Setup for *SCOPE* **and** *Resynthesis + SCOPE***.** We obtain the *SCOPE* implementation from [3] and the *Resynthesis + SCOPE* implementation from [39]. We configure both tools as follows. First, since *SCOPE* cannot handle any loops in the designs, e.g., introduced through sequential feedback operations on FFs, we convert any FFs into pseudo PIs/POs. Note that this is the technically correct handling of sequential designs as unrolled combinational designs, and doing so is common practice in the literature. Also note that, for the FFs building up the shift register storing the key, we designate all related pseudo PIs as individual key-bit PIs. Second, we utilize the *Resynthesis + SCOPE* framework with its default settings; the framework then automatically handles all required steps, like conversion from *Verilog* to *bench* netlists as required for *SCOPE*, etc.

Metrics. We consider the following established metrics: *accuracy (AC)*, *precision (PC)*, and *key prediction accuracy (KPA)*. AC measures the percentage of correctly deciphered key-bits, i.e., $(k_{correct}/k_{total})$. PC measures the correctly deciphered key-bits, optimistically considering every *X*/undeciphered value as a correct guess, i.e., $((k_{correct} + k_X)/k_{total})$. KPA measures the correctly deciphered key-bits over the entire prediction set, i.e., $(k_{correct}/(k_{total} - k_X))$. Furthermore, we consider the *constant propagation effect (COPE)*, which specifically measures the vulnerability against the *SCOPE* attack; in short, COPE = 0% means the attack fails entirely. All metrics are reported in percentage.

Results for *MuxLink***.** The AC metrics in Table 5 show that *MuxLink* can predict only 20.85% of all key-bits correctly, on average. Interestingly, it holds across the various benchmarks (except for *SPARX*) that, the larger is the key length, the higher is *MuxLink*'s accuracy. However, this does *not* imply that designs hardened with larger *TroLLoc-MUX* keys would be more prone to second-order attacks. This is because, at the same time, (1) the average KPA is 53.17%, implying that, overall, *MuxLink* is not performing much better than random guessing, and (2) average KPA remains rather steady across designs. In other words, for designs with larger *TroLLoc-MUX* keys, while *MuxLink* succeeds to correctly infer a larger share of key-bits, it also *incorrectly* infers an equally larger share of key-bits. Thus, the scalability of *TroLLoc-MUX* 's robustness against *MuxLink* is confirmed.

Table 5 Results for *MuxLink* on *TroLLoc-MUX* and *D-MUX*

Design	***TroLLoc-MUX***				***D-MUX***			
	AC	*PC*	*KPA*	*#(X)*	*AC*	*PC*	*KPA*	*#(X)*
AES_1	28.7%	71.4%	50.2%	512	78.1%	79.9%	79.5%	21
AES_2	**28.1%**	**75.0%**	**52.9%**	**564**	**80.4%**	**81.7%**	**81.4%**	**15**
AES_3	08.1%	90.0%	45.0%	361	88.4%	89.1%	89.0%	3
Camellia	**26.1%**	**73.8%**	**49.9%**	**606**	**90.6%**	**92.6%**	**92.4%**	**25**
CAST	37.4%	66.0%	52.4%	449	93.7%	94.3%	94.2%	7
MISTY	**33.3%**	**65.8%**	**49.3%**	**395**	**97.8%**	**98.0%**	**98.0%**	**3**
openMSP430_1	23.5%	76.6%	50.1%	646	NA	NA	NA	NA
openMSP430_2	**09.3%**	**90.4%**	**49.5%**	**440**	**66.8%**	**69.9%**	**69.0%**	**17**
PRESENT	11.6%	95.0%	70.0%	201	NA	NA	NA	NA
SEED	**37.0%**	**64.0%**	**50.7%**	**435**	**97.3%**	**97.7%**	**97.7%**	**6**
SPARX	05.3%	94.9%	51.4%	2,312	NA	NA	NA	NA
TDEA	**01.8%**	**99.0%**	**66.6%**	**208**	**NA**	**NA**	**NA**	**NA**

AC: accuracy; PC: precision; KPA: key prediction accuracy; #(X): nr. of undeciphered key-bits; NA: locking using the script in [5] fails.

Table 6 Results for *SCOPE* on *TroLLoc-MUX*

Design	*AC(1)*	*PC(1)*	*KPA(1)*	*AC(2)*	*PC(2)*	*KPA(2)*	*COPE*
AES_1	21.10%	79.23%	50.39%	20.76%	78.89%	49.60%	18.45%
AES_2	**22.46%**	**78.03%**	**50.56%**	**21.96%**	**77.53%**	**49.43%**	**17.87%**
AES_3	9.52%	90.92%	51.21%	9.07%	90.47%	48.78%	10.32%
Camellia	**11.40%**	**91.42%**	**57.08%**	**8.57%**	**88.59%**	**42.91%**	**0.10%**
CAST	12.40%	87.91%	50.64%	12.08%	87.59%	49.35%	0.05%
MISTY	**17.77%**	**81.15%**	**48.53%**	**18.84%**	**82.22%**	**51.46%**	**0.35%**
openMSP430_1	16.09%	84.07%	50.25%	15.92%	83.90%	49.74%	0.24%
openMSP430_2	**13.25%**	**88.76%**	**54.13%**	**11.23%**	**86.74%**	**45.86%**	**0.18%**
PRESENT	1.24%	94.60%	18.75%	5.39%	98.75%	81.25%	2.79%
SEED	**11.97%**	**87.22%**	**48.37%**	**12.77%**	**88.02%**	**51.62%**	**0.05%**
SPARX	4.02%	96.51%	53.60%	3.48%	95.97%	46.39%	0.14%
TDEA	**0.46%**	**100%**	**100%**	**0.00%**	**99.53%**	**0.00%**	**0.01%**

AC: accuracy; PC: precision; KPA: key prediction accuracy; COPE: constant propagation effect [3].

For the sake of putting *MuxLink*'s performance on *TroLLoc-MUX* into some context for prior art, we have implemented *D-MUX* [5, 69] and configured it for for the same key length as *TroLLoc-MUX* (Table 3). The corresponding results, also shown in Table 5, clearly demonstrate that *TroLLoc-MUX* is superior to *D-MUX* , across all metrics.

Results for *SCOPE***.** The results in Table 6 show that, on average, *SCOPE* can predict only 11.81% / 11.67% of all key-bits correctly.[12] The average KPA of 52.79% / 47.20% shows that *SCOPE* is largely equivalent to random guessing as well. Note that we observe similar trends for key length and accuracy with *SCOPE* as with *MuxLink*; the scalability of *TroLLoc-MUX* 's robustness against *SCOPE* is confirmed. Also note that *SCOPE* can correctly decipher only a single bit for the benchmark TDEA, which trivially results in $PC(1) = KPA(1) = 100\%$ and $PC(2) \approx 100\%, KPA(2) = 0\%$.

Results for *Resynthesis* + *SCOPE***.** The results in Table 7 show that *Resynthesis* + *SCOPE* can predict 26.95% / 27.31% of all key-bits correctly, on average. While these accuracy results are superior to those of *SCOPE* alone—as expected—this does *not* mean that *TroLLoc-MUX* is more prone to second-order attacks by *Resynthesis* + *SCOPE*. This is because the average KPA of 49.16% / 50.83% shows that *Resynthesis* + *SCOPE* still does not perform any better than random guessing. Similar trends for key length and accuracy are observed again; the scalability of *TroLLoc-MUX* 's robustness against *Resynthesis* + *SCOPE* is confirmed as well.

[12] *SCOPE*predicts two mutually complementary keys. The related metrics are reported as 'AC(1)' versus 'AC(2)', etc., respectively.

6.4 Case Study 4: Second-Order Attacks on *TroLLoc-XOR*

Setup for *OMLA* **and Metrics.** We obtain the *OMLA* implementation from [6] and revise it as follows. First, note that, in its original implementation, when given a locked design to attack, *OMLA* seeks to lock further parts of that design and then utilizes those newly locked parts as training data. For *TroLLoc-XOR*, however, any further locking is impractical, as the protected layouts are already locked as much as possible, exhibiting ultra-high utilization and/or very few relevant nets remain unlocked. Thus, for this case study, we train *OMLA* via a classical leave-one-out scheme, considering all respectively remaining, locked designs from the ISPD'22 suite. Second, based on the findings for *TroLLoc-XOR* without gate balancing (first set of results further below), we revise the cell selection described in Algorithm 1 as follows. For each gate type, we track the number of locked instances. During the iterative selection and locking procedure, we then balance each type with its complementary counterpart, e.g., `NAND2` and `AND2`. Before each round of cell selection, we temporarily ignore a cell if locking that cell would break the balance. As for metrics, note that, unlike *SCOPE* or *Muxlink*, *OMLA* always provides an inference for all key-bits. Thus, $k_X = 0$ and $AC = PC = KPA$.

Results for *TroLLoc-XOR* without Gate Balancing. The results in Table 8 show that, without gate balancing being applied for *TroLLoc-XOR*, *OMLA* can achieve an average KPA of 69%, which is notably better than random guessing. This is due to the fact that the composition of locked gates can be unbalanced in terms of

Table 7 Results for *Resynthesis + SCOPE* on *TroLLoc-MUX*

Design	*AC(1)*	*PC(1)*	*KPA(1)*	*AC(2)*	*PC(2)*	*KPA(2)*	*Avg. COPE*
AES_1	32.77%	68.14%	50.70%	31.85%	67.22%	49.29%	17.89%
AES_2	**33.94%**	**65.97%**	**49.93%**	**34.02%**	**66.05%**	**50.06%**	**17.13%**
AES_3	12.24%	86.16%	46.95%	13.83%	87.75%	53.04%	9.96%
Camellia	**34.46%**	**68.13%**	**51.95%**	**31.86%**	**65.53%**	**48.04%**	**0.23%**
CAST	41.92%	58.96%	50.53%	41.03%	58.07%	49.46%	0.17%
MISTY	**40.49%**	**57.94%**	**49.05%**	**42.05%**	**59.50%**	**50.94%**	**0.53%**
openMSP430_1	27.99%	72.41%	50.36%	27.58%	72.00%	49.63%	0.65%
openMSP430_2	**23.02%**	**81.03%**	**54.82%**	**18.96%**	**76.97%**	**45.17%**	**0.95%**
PRESENT	18.25%	75.51%	42.71%	24.48%	81.74%	57.28%	2.40%
SEED	**40.63%**	**58.62%**	**49.54%**	**41.37%**	**59.36%**	**50.45%**	**0.16%**
SPARX	7.39%	92.40%	49.35%	7.59%	92.60%	50.64%	0.16%
TDEA	**10.28%**	**86.91%**	**44.00%**	**13.08%**	**89.71%**	**56.00%**	**0.60%**

AC: accuracy; PC: precision; KPA: key prediction accuracy; COPE: constant propagation effect [3].

Table 8 Results for *OMLA* on *TroLLoc-XOR*

Design	*TroLLoc-XOR*, Unbalanced			*TroLLoc-XOR*, Balanced		
	KL	*Utils*	*KPA*	*KL*	*Utils*	*KPA*
AES_1	1,200	96.1%	79.1%	1,273	95.0%	51.5%
AES_2	**1,149**	**95.0%**	**78.9%**	**1,215**	**94.3%**	**52.1%**
AES_3	416	95.5%	63.7%	416	95.6%	52.6%
Camellia	**1,348**	**98.7%**	**75.6%**	**1,051**	**86.8%**	**48.8%**
CAST	1,804	94.6%	78.2%	1,707	96.2%	68.4%
MISTY	**1,315**	**93.9%**	**78.9%**	**1,172**	**93.8%**	**67.6%**
openMSP430_1	1,337	98.7%	70.2%	1,154	91.2%	56.8%
openMSP430_2	**582**	**97.4%**	**66.5%**	**582**	**97.2%**	**55.0%**
PRESENT	255	97.4%	47.8%	208	88.7%	52.9%
SEED	**1,802**	**94.1%**	**78.5%**	**1,618**	**92.4%**	**66.9%**
SPARX	2,746	99.0%	56.6%	2,741	98.9%	48.2%
TDEA	**226**	**99.0%**	**54.0%**	**226**	**99.0%**	**53.5%**

KL: key length (bits), accounting both for locked assets and other locked cell instances; Utils: utilization after physical synthesis; KPA: key prediction accuracy.

the original and unlocked gate's type. For example, in *AES_1*, 441 `NAND2` gates are locked while there are only 37 `AND2` gates locked for *TroLLoc-XOR* without balancing applied. Such trends occur in other designs as well, from which *OMLA* learns about the majority of the original gates' types. The same challenge has been discussed in [69]: once there is a notable imbalance in the distribution of gate-type instances, locking schemes start to leak structural information, which requires further efforts to mitigate. Toward that end, as indicated, we employ gate balancing. Related results are discussed next.

Results for *TroLLoc-XOR* with Gate Balancing. The results in Table 8 show that, with gate balancing applied for *TroLLoc-XOR*, *OMLA* can only achieve an average KPA of 56.19%, which is closer to random guessing, namely by 12.81 percentage points (pps). For the previous example of *AES_1*, KPA is even reduced by 27.60 pps.

We find that *TroLLoc-XOR* with gate balancing can be limited, by the original gate composition, in terms of achievable utilization and key length. In fact, key lengths are 5.76% shorter and utilization numbers are 2.53 pps lower on average. To mitigate this limitation, efforts would be required from the earlier design stages, prior to physical synthesis, namely logic synthesis. More specifically, gate balancing could become another optimization objective, in addition to the classical power, performance, and area objectives. Note that some prior art for security-aware logic synthesis exist, e.g., to reduce power side-channel information leakage by means of synthesizing differential logic [36], but not for security closure against Trojan insertion. Such efforts are left for future work.

7 Conclusion

We propose a novel scheme for proactive, pre-silicon Trojan prevention by means of integrated logic locking and layout hardening, all in a real-world setup for IC design. Our work successfully addresses three key challenges for Trojan countermeasures: (i) ours remains robust against potential second-order attacks, i.e., adversaries seeking to bypass the layout-level defense before actual Trojan insertion, (ii) ours is effective as in protecting layouts in general and security assets in particular against real-world insertion of various Trojans, and (iii) ours is efficient as in achieving favorable security-versus-cost trade-offs, by careful integration of our techniques into commercial-grade IC tools.

Through elaborate case studies, we demonstrate these claims and also find further important aspects. More specifically, we find that *TroLLoc-XOR* is slightly more effective against actual Trojan insertion, whereas *TroLLoc-MUX* is slightly more effective against ML-based attacks on locking that could be utilized for second-order attacks. This implies that proactive protection against HTs depends on the attacker's capabilities and the layout-level intricacies; this is somewhat expected and aligns with recent other works as well [41, 73]. Thus, a security-aware design methodology like ours is essential for protection of sensitive ICs.

Statements and Declarations

Funding This work is partially sponsored by the Chinese University of Hong Kong and New York University/New York University Abu Dhabi (NYU/NYUAD) Center for Cyber Security (CCS).

Competing interests The authors declare no competing interests.

Author Contributions F. W., Q.W., L.A., J.K. wrote the main manuscript text and figures. B.F., S.J., X.Z. helped to implement part of the proposed flow. O.S., T.H., F.Y. reviewed the manuscript.

Data Availability Data is provided within the manuscript or supplementary information files.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

References

1. Adee S (2008) The hunt for the kill switch. Spectrum 45(5). https://doi.org/10.1109/MSPEC.2008.4505310
2. Agrawal D et al (2007) Trojan detection using IC fingerprinting. In: Proceedings of SP. https://doi.org/10.1109/SP.2007.36
3. Alaql A et al (2021) SCOPE: Synthesis-based constant propagation attack on logic locking. Trans VLSI 29(8)
4. Almeida F et al (2023) Resynthesis-based attacks against logic locking. In: Proceedings of ISQED https://doi.org/10.1109/ISQED57927.2023.10129403
5. Alrahis L et al (2022) MuxLink: Circumventing learning-resilient MUX-locking using graph neural network-based link prediction. In: Proceedings of DATE
6. Alrahis L et al (2022) OMLA: An oracle-less machine learning-based attack on logic locking. TCS 69(3)
7. Ba PS et al (2015) Hardware Trojan prevention using layout-level design approach. In: European Conference on Circuit Theory and Design
8. Ba PS et al (2016) Hardware trust through layout filling: A hardware Trojan prevention technique. In: Proceedings of ISVLSI
9. Baek K et al (2022) Pin accessibility and routing congestion aware DRC hotspot prediction using graph neural network and u-net. In: Proceedings of ICCAD. https://doi.org/10.1145/3508352.3549346
10. Basak A et al (2017) Security assurance for system-on-chip designs with untrusted IPs. Trans Inf Forens Sec 12(7)
11. Chakraborty P et al (2018) SAIL: Machine learning guided structural analysis attack on hardware obfuscation. In: Proceedings of AHOST
12. Chen X et al (2018) Hardware Trojan detection in third-party digital intellectual property cores by multilevel feature analysis. TCAD
13. Clark LT et al (2016) ASAP7: A 7-nm FinFET predictive process design kit. Microelectron J 53:105–115. https://doi.org/10.1016/j.mejo.2016.04.006
14. Deng D et al (2020) Novel design strategy toward A2 Trojan detection based on built-in acceleration structure. TCAD 39(12)
15. Dong C et al (2020) Hardware Trojans in chips: A survey for detection and prevention. Sensors 20(18)
16. Dupuis S et al (2014) A novel hardware logic encryption technique for thwarting illegal overproduction and hardware Trojans. In: Proceedings of IOLTS
17. Eslami M et al (2023) Benchmarking advanced security closure of physical layouts: ISPD 2023 contest. In: Proceedings of ISPD. https://doi.org/10.1145/3569052.3578924. https://wp.nyu.edu/ispd23_contest/
18. Eslami M et al (2023) SALSy: Security-aware layout synthesis
19. Eslami M et al (2024) SCARF: Securing chips with a robust framework against fabrication-time hardware Trojans
20. Fern N et al (2017) Detecting hardware Trojans in unspecified functionality through solving satisfiability problems. In: Proceedings of ASP-DAC
21. Ghandali S et al (2020) Side-channel hardware Trojan for provably-secure SCA-protected implementations. Trans VLSI 28(6):1435–1448. https://doi.org/10.1109/TVLSI.2020.2982473
22. Gohil V et al (2022) ATTRITION: Attacking static hardware Trojan detection techniques using reinforcement learning. In: Proceedings of CCS. https://doi.org/10.1145/3548606.3560690
23. Guimarães LA et al (2017) Detection of layout-level Trojans by monitoring substrate with preexisting built-in sensors. In: Proceedings of ISVLSI

24. Guo G et al (2023) ASSURER: A PPA-friendly security closure framework for physical design. In: Proceedings of ASP-DAC
25. Guo X et al (2019) QIF-Verilog: Quantitative information-flow based hardware description languages for pre-silicon security assessment. In: Proceedings of HOST
26. Hasegawa K et al (2017) Trojan-feature extraction at gate-level netlists and its application to hardware-Trojan detection using random forest classifier. In: Proceedings of ISCAS. https://doi.org/10.1109/ISCAS.2017.8050827
27. Hepp A et al (2022) A pragmatic methodology for blind hardware Trojan insertion in finalized layouts. In: Proceedings of ICCAD. https://doi.org/10.1145/3508352.3549452
28. Hossein-Talaee H, Jahanian A (2017) Layout vulnerability reduction against Trojan insertion using security-aware white space distribution. In: Proceedings of ISVLSI
29. Hou Y et al (2018) R2D2: Runtime reassurance and detection of A2 Trojan. In: Proceedings of HOST
30. Hsu JW et al (2023) Security-aware physical design against Trojan insertion, frontside probing, and fault injection attacks. In: Proceedings of ISPD. https://doi.org/10.1145/3569052.3571876
31. Hu W et al (2020) An overview of hardware security and trust: Threats, countermeasures and design tools. TCAD
32. Imeson F et al (2013) Securing computer hardware using 3D integrated circuit (IC) technology and split manufacturing for obfuscation. In: Proceedings of USENIX Security Symposium
33. Jacob N et al (2014) Hardware Trojans: Current challenges and approaches. Comp Dig Techs 8(6).https://doi.org/10.1049/iet-cdt.2014.0039
34. Kim LW, Villasenor JD (2011) A system-on-chip bus architecture for thwarting integrated circuit Trojan horses. Trans, VLSI
35. Knechtel J (2021) Hardware security for and beyond CMOS technology. In: Proceedings of ISPD. https://doi.org/10.1145/3439706.3446902
36. Knechtel J et al (2020) Towards secure composition of integrated circuits and electronic systems: On the role of EDA. In: Proceedings of DATE
37. Knechtel J et al (2021) Security closure of physical layouts. In: Proceedings of ICCAD
38. Knechtel J et al (2022) Benchmarking security closure of physical layouts: ISPD 2022 contest. In: Proceedings of ISPD
39. Knechtel J et al (2023) Resynthesis + SCOPE Setup. https://github.com/DfX-NYUAD/resynthesis_attack
40. Knechtel J et al (2024) Github: Trojan insertion versus layout defenses for modern ICs: Red-versus-blue teaming in a competitive community effort. https://github.com/DfX-NYUAD/Trojan-Insertion-versus-Layout-Defenses
41. Knechtel J et al (2024) Trojan insertion versus layout defenses for modern ICs: Red-versus-blue teaming in a competitive community effort. Proc TCHES 2025(1):37–77. https://doi.org/10.46586/tches.v2025.i1.37-77
42. Lashen H et al (2023) TrojanSAINT: Gate-level netlist sampling-based inductive learning for hardware Trojan detection. In: Proceedings of ISCAS
43. Liu K, Yang H, Ma Y, Tan B, Yu B, Young EF, Karri R, Garg S (2020) Adversarial perturbation attacks on ml-based cad: A case study on cnn-based lithographic hotspot detection. ACM Trans Design Autom Electron Syst (TODAES) 25(5):1–31
44. Marcelli A et al (2017) An evolutionary approach to hardware encryption and trojan-horse mitigation. In: Proceedings of DATE
45. Mardani Kamali H et al (2018) LUT-Lock: A novel LUT-based logic obfuscation for FPGA-bitstream and ASIC-hardware protection. In: Proceedings of ISVLSI. https://doi.org/10.1109/ISVLSI.2018.00080
46. Muehlberghuber M et al (2013) Red team vs. blue team hardware Trojan analysis: Detection of a hardware Trojan on an actual ASIC. In: Proceedings of international workshop on hardware and architectural support for security and privacy. https://doi.org/10.1145/2487726.2487727
47. NanGate FreePDK45 Open Cell Library (2011). http://www.nangate.com/?page_id=2325
48. Narasimhan S et al (2013) Hardware Trojan detection by multiple-parameter side-channel analysis. Trans Comp 62(11)
49. Perez T, Pagliarini S (2022) Hardware Trojan insertion in finalized layouts: From methodology to a silicon demonstration. TCAD. https://doi.org/10.1109/TCAD.2022.3223846
50. Puschner E et al (2023) Red team vs. blue team: A real-world hardware Trojan detection case study across four modern CMOS technology generations. In: Proceedings of SP. https://doi.org/10.1109/SP46215.2023.00044
51. Rajendran J et al (2015) Fault analysis-based logic encryption. Trans Comp 64(2)
52. Ravi P et al (2019) Security is an architectural design constraint. Microprocess. Microsyst 68(C)
53. Rostami M et al (2014) A primer on hardware security: Models, methods, and metrics. Proceedings of IEEE 102(8)
54. Roy JA et al (2010) Ending piracy of integrated circuits. Computer 43(10). https://doi.org/10.1109/MC.2010.284
55. Salmani H et al (2016) Vulnerability analysis of a circuit layout to hardware Trojan insertion. Trans Inf Forens Sec 11(6)
56. Samimi MS et al (2016) Hardware enlightening: No where to hide your hardware Trojans! In: Proceedings of IOLTS
57. Shi Q et al (2018) Obfuscated built-in self-authentication with secure and efficient wire-lifting. TCAD 38(11)
58. Sigl G (2011) Keynote address: Design of secure systems - where are the EDA tools? In: Proceedings of ICCAD
59. Skorobogatov S, Woods C (2012) Breakthrough silicon scanning discovers backdoor in military chip. In: Proceedings of TCHES
60. Subramanyan P et al (2015) Evaluating the security of logic encryption algorithms. In: Proceedings of HOST. https://doi.org/10.1109/HST.2015.7140252
61. Sugawara T et al (2015) Reversing stealthy dopant-level circuits. J Cryptogr Eng 5(2)
62. Trippel T et al (2020) ICAS: an extensible framework for estimating the susceptibility of IC layouts to additive Trojans. In: Proceedings of SP
63. Trippel T et al (2023) T-TER: Defeating A2 Trojans with targeted tamper-evident routing. In: Proceedings of CCS. https://doi.org/10.1145/3579856.3582837
64. TroLLoc artifacts: Baseline and protected layouts (2024). https://drive.google.com/file/d/17ysAYnoPgFuy0Yj969dTGUWZVrIT7PmW/view?usp=sharing
65. Vashishtha V et al (2017) ASAP7 predictive design kit development and cell design technology co-optimization: Invited paper. In: Proceedings of ICCAD, pp 992–998. https://doi.org/10.1109/ICCAD.2017.8203889
66. Vashistha N et al (2018) Trojan scanner: Detecting hardware Trojans with rapid SEM imaging combined with image processing and machine learning. In: ISTFA
67. Vijayan A et al (2020) Runtime identification of hardware Trojans by feature analysis on gate-level unstructured data and anomaly detection. TODAES 25(4)
68. Šišejković D et al (2019) Control-lock: Securing processor cores against software-controlled hardware Trojans. In: Proceedings of GLSVLSI
69. Šišejković D et al (2022) Deceptive logic locking for hardware integrity protection against machine learning attacks. TCAD 41(6)

70. Wahby RS et al (2016). Verifiable ASICs. Proc SP. https://doi.org/10.1109/SP.2016.51
71. Wang F et al (2023) Security closure of IC layouts against hardware Trojans. In: Proceedings of ISPD. https://doi.org/10.1145/3569052.3571878
72. Wei X et al (2023) GDSII-Guard: ECO anti-Trojan optimization with exploratory timing-security trade-offs. In: Proceedings of DAC. https://doi.org/10.1109/DAC56929.2023.10247697
73. Wei X et al (2024) Rethinking IC layout vulnerability: Simulation-based hardware Trojan threat assessment with high fidelity. In: Proceedings of SP. https://doi.org/10.1109/SP54263.2024.00160
74. Wu TF et al (2016) TPAD: Hardware Trojan prevention and detection for trusted integrated circuits. TCAD 35(4). https://doi.org/10.1109/TCAD.2015.2474373
75. Xiao K et al (2014) A novel built-in self-authentication technique to prevent inserting hardware Trojans. TCAD 33(12)
76. Xiao K et al (2016) Hardware Trojans: Lessons learned after one decade of research. TODAES 22(1)
77. Yang K et al (2016) A2: Analog malicious hardware. In: Proceedings of SP
78. Yasin M et al (2016) Activation of logic encrypted chips: Pre-test or post-test? In: Proceedings of DATE
79. Yasin M et al (2020) Trustworthy Hardware Design: Combinational Logic Locking Techniques. Springer

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.